Particle Filter Project

Özge Bülbül
Student ID: 2220765008

Sümeyra Koç
Student ID: 2210765020

Zeynep Yıldız
Student ID: 2210765033

June 2026

1 Introduction and Theoretical Background

Robot localization is a fundamental problem in mobile robotics. The objective is to estimate the pose of a robot, represented by its position and orientation, while accounting for uncertainty arising from sensor noise and imperfect motion measurements. In this project, a Particle Filter (PF) was implemented to estimate the pose of a mobile robot navigating in a simulated environment containing multiple AR tags.

The Particle Filter is a probabilistic localization method based on the principles of Bayesian Filtering. Rather than maintaining a single estimate of the robot pose, the algorithm represents the belief state as a set of weighted particles. Each particle corresponds to a possible robot pose hypothesis and its associated probability.

Bayesian Filtering recursively estimates the posterior probability distribution of the robot state according to

$$p(x_t|z_{1:t}, u_{1:t})$$

where x_t denotes the robot state, $z_{1:t}$ represents all sensor observations up to time t, and $u_{1:t}$ denotes the control or motion inputs.

The Bayesian filtering process consists of three fundamental probabilistic concepts:

- **Prior:** The predicted belief about the robot state before incorporating the current sensor measurement.
- **Likelihood:** The probability of obtaining the current observation assuming a specific robot state.
- **Posterior:** The updated belief obtained after combining the prior and the likelihood according to Bayes' theorem.

The Particle Filter approximates this recursive Bayesian estimation through three main steps.

1.1 Prediction Step

During the prediction step, the robot's odometry measurements are used to propagate all particles according to the motion model. Random Gaussian noise is added to account for uncertainty in the robot's movement. This stage corresponds to computing the prior distribution.

1.2 Update (Correction) Step

When an AR tag observation is received, the likelihood of that observation is evaluated for every particle. Particles that better explain the observation receive higher weights, while particles inconsistent with the observation receive lower weights. This stage incorporates sensor information and produces the posterior distribution.

1.3 Resampling Step

Over time, many particles obtain negligible weights and contribute little to the state estimate. To avoid particle degeneracy, a resampling procedure is performed. Particles with high weights are duplicated while particles with low weights are discarded. This allows computational resources to be concentrated in regions with high posterior probability.

Consequently, the Particle Filter provides a practical approximation of Bayesian filtering by representing probability distributions using a finite set of weighted samples. As the robot moves and receives additional observations, the particle cloud gradually converges toward the true robot pose.

2 Simulation Setup

The localization system was developed and evaluated using **Gazebo Harmonic** as the simulation environment and **ROS 2 Jazzy** as the robotics middleware. The simulation environment provided realistic physics, sensor simulation, and communication interfaces required for testing the Particle Filter localization algorithm.

2.1 Environment Configuration

A rectangular room with dimensions of $\mathbf{4\,m} \times \mathbf{6\,m}$ was created for the experiments. Since Gazebo Harmonic does not provide the same ease of room construction as Gazebo Classic, a corner of a built-in warehouse environment was utilized and additional wall boundaries were created using cube objects. This approach enabled the creation of a controlled indoor environment suitable for visual landmark-based localization.

Several ArUco tags were placed at known positions throughout the environment. The exact tag arrangement is illustrated in Figure 1. The known locations of these tags constitute the map information used by the localization system.

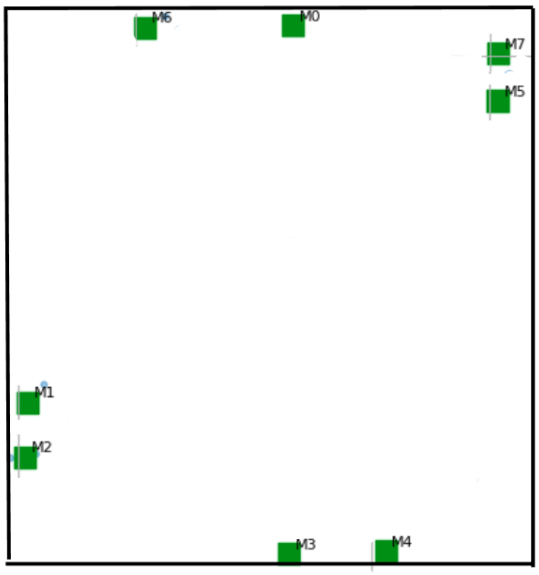

Figure 1: Layout of the ArUco tags used in the simulation environment.

2.2 Robot Platform

The mobile robot is a differential-drive platform consisting of two powered front wheels and a passive caster wheel at the rear. The wheel separation was configured as 0.20m, and each wheel has a radius of 0.066m.

Robot motion and odometry generation were simulated using the `gz::sim::systems::DiffDrive` plugin. This plugin computes the robot's motion based on wheel rotations and publishes odometry information as well as the corresponding transforms between the `odom` and `base_footprint` coordinate frames.

Wheel joint states were published using the `gz::sim::systems::JointStatePublisher` plugin, enabling integration with the ROS 2 transform tree and other localization components.

A visualization of the robot platform is provided in Figure 2.

2.3 Camera Configuration

The robot is equipped with a forward-facing RGB camera mounted at the front of the chassis. The camera sensor was configured according to the parameters listed in Table 1.

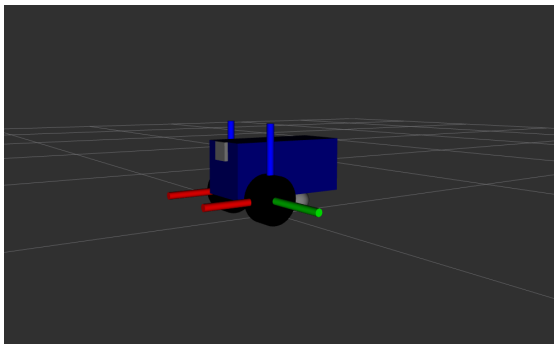

Figure 2: Differential-drive robot used in the simulation.

Table 1: Camera parameters used in simulation.

Parameter	**Value**
Image resolution	640×480 pixels
Horizontal field of view	1.3962634 rad ($\approx 80°$)
Near clipping plane	0.1 m
Far clipping plane	15 m
Update rate	20 Hz
Noise model	Gaussian ($\sigma = 0.007$)

The camera publishes image data on the `/camera/image_raw` topic and calibration information on the `/camera/camera_info` topic. To better approximate real-world sensing conditions, Gaussian noise was added to the generated images.

2.4 Simulation Overview

Figure 3 presents the complete simulation environment used throughout the experiments. The figure shows the warehouse-based room configuration, robot platform, and ArUco marker placements.

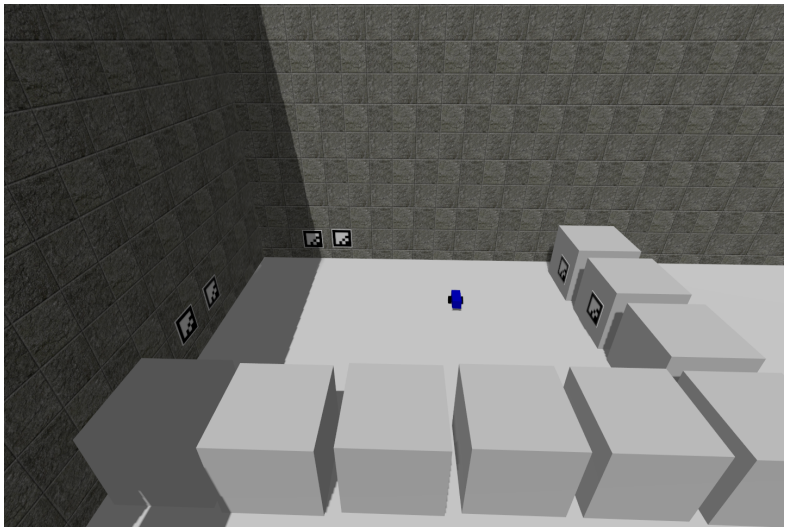

Figure 3: Overview of the Gazebo Harmonic simulation environment.

3 Motion Model (Prediction Step)

This section presents the motion model used during the prediction stage of the Particle Filter localization system.

The robot's motion is propagated using an odometry-based motion model. At each time step, the raw displacement deltas ($\Delta x_{odom}, \Delta y_{odom}, \Delta\theta$) are captured from the `/odom` topic, representing the displacement between the current pose and the previous pose in the odometry frame. Since these deltas are given in the global odometry frame, they are first transformed into the robot's local coordinate frame relative to the previous orientation (θ_{prev}) using a 2D rotation matrix:

$$\Delta x_{local} = \cos(-\theta_{prev})\Delta x_{odom} - \sin(-\theta_{prev})\Delta y_{odom} \tag{1}$$

$$\Delta y_{local} = \sin(-\theta_{prev})\Delta x_{odom} + \cos(-\theta_{prev})\Delta y_{odom} \tag{2}$$

To account for non-systematic uncertainties present in real-world robotic systems, such as wheel slippage, floor imperfections or odometry integration drift, zero-mean Gaussian noise is added to the local displacements and the angular heading change before updating the state of each particle. The noisy motion components ($\hat{\Delta x}, \hat{\Delta y}, \hat{\Delta\theta}$) are sampled as follows:

$$\hat{\Delta x} = \Delta x_{local} + \mathcal{N}(0, \sigma^2_{forward}) \tag{3}$$

$$\hat{\Delta y} = \Delta y_{local} + \mathcal{N}(0, \sigma^2_{lateral}) \tag{4}$$

$$\hat{\Delta\theta} = \Delta\theta + \mathcal{N}(0, \sigma^2_{\theta}) \tag{5}$$

where the noise parameters are configured empirically based on the simulation environment constraints to maintain a representative spread of hypotheses:

- Forward translation noise standard deviation: $\sigma_{forward} = 0.035m$
- Lateral translation noise standard deviation: $\sigma_{lateral} = 0.025m$
- Rotational heading noise standard deviation: $\sigma_{\theta} = 0.020rad$

Each individual particle i updates its pose hypothesis $[x^{(i)}, y^{(i)}, \theta^{(i)}]^T$ by projecting these noisy local displacements into its own orientation frame. The kinematic transition equations are defined as:

$$x_t^{(i)} = x_{t-1}^{(i)} + \hat{\Delta x}\cos(\theta_{t-1}^{(i)}) - \hat{\Delta y}\sin(\theta_{t-1}^{(i)}) \tag{6}$$

$$y_t^{(i)} = y_{t-1}^{(i)} + \hat{\Delta x}\sin(\theta_{t-1}^{(i)}) + \hat{\Delta y}\cos(\theta_{t-1}^{(i)}) \tag{7}$$

$$\theta_t^{(i)} = wrap\_angle(\theta_{t-1}^{(i)} + \hat{\Delta\theta}) \tag{8}$$

where the $wrap\_angle(\cdot)$ function normalizes the orientation within the $[-\pi, \pi]$ range using the $atan2$ function to prevent angular discontinuities during multi-turn executions.

Finally, a boundary clamping mechanism is enforced to ensure environmental realism and maintain computational efficiency by filtering out physically

impossible particle trajectories. The particles are strictly bounded within the known room coordinates:

$$x_t^{(i)} = \max(\min(x_t^{(i)}, x_{max}), x_{min}) \tag{9}$$

$$y_t^{(i)} = \max(\min(y_t^{(i)}, y_{max}), y_{min}) \tag{10}$$

where the world boundaries matching the warehouse environment configurations are defined as $x_{min} = -16.0m$, $x_{max} = -9.0m$, $y_{min} = -28.0m$, and $y_{max} = -19.0m$. Particles that are predicted to cross these boundaries are constrained to the edges, preventing the particle cloud from expanding into unmapped regions outside the simulation room and ensuring that invalid state hypotheses do not survive.

4 Sensor Model (Update Step)

4.1 Observation Likelihood

The sensor model is responsible for evaluating how well each particle explains the observations obtained from the robot's camera. In the simulation environment, the robot detects AR tags placed on the walls of the room. However, all tags share the same identifier, which introduces ambiguity regarding the identity of the observed tag.

For each detected AR tag, the camera provides a relative measurement consisting of:

- Range r: the distance between the robot and the observed tag,
- Bearing ϕ: the relative angle of the tag with respect to the robot's heading.

For a given particle x, the expected range and bearing to every tag in the map are computed using the known tag coordinates. Assuming Gaussian measurement noise, the likelihood associated with a particular tag hypothesis is calculated as

$$p(z|x, tag_i) = \exp\left(-\frac{1}{2}\left(\frac{r_m - r_i}{\sigma_r}\right)^2\right) \cdot \exp\left(-\frac{1}{2}\left(\frac{\phi_m - \phi_i}{\sigma_\phi}\right)^2\right),$$

where

- r_m and ϕ_m are the measured range and bearing,
- r_i and ϕ_i are the expected values predicted by the particle for tag i,
- σ_r and σ_ϕ represent the range and bearing noise parameters.

A key requirement of this project is that all AR tags have identical IDs. Therefore, the robot cannot determine which specific tag generated the observation. Instead of selecting the nearest tag, the sensor model evaluates every tag as a possible source of the observation.

The overall observation likelihood is computed using a multi-hypothesis formulation:

$$p(z|x) = \sum_{i=1}^{N} p(z|x, tag_i),$$

where N denotes the total number of AR tags in the environment.

This approach allows multiple localization hypotheses to coexist during the early stages of localization. Several particle clusters may simultaneously receive significant likelihood values because different regions of the environment can produce similar observations. As additional measurements become available, incorrect hypotheses gradually receive lower weights and disappear through the resampling process.

After computing the likelihood, the weight of each particle is updated according to

$$w_t^{(k)} = w_{t-1}^{(k)} \cdot p(z_t|x_t^{(k)}),$$

where $w_t^{(k)}$ denotes the weight of particle k. The updated weights are then normalized so that their sum equals one.

Through this multi-hypothesis sensor model, the particle filter is able to handle tag ambiguity while still converging toward the correct robot pose as more observations are collected.

4.2 Resampling and Jittering

To prevent particle degeneracy after weight normalization, systematic resampling is triggered dynamically whenever the Effective Sample Size (ESS) drops below 50% of the total particle count. Following the resampling step, a positional roughening (jitter) process is applied by adding small zero-mean Gaussian noise ($\sigma_{pos} = 0.01\,\text{m}$ and $\sigma_\theta = 0.005\,\text{rad}$) to each particle's pose. This mechanism maintains spatial diversity within the particle cloud and ensures the filter can recover from unexpected tracking or odometry drift.

5 Experimental Results and Discussion

5.1 Initial Spread

At initialization, particles are distributed throughout the environment according to the chosen localization strategy. In this project, particles are uniformly distributed across the entire map bounds at initialization to solve the global localization problem.

Figure 4 illustrates the initial particle distribution. Since no prior information about the robot pose is assumed, particles are randomly sampled across the complete environment. As a result, the particle cloud exhibits a nearly uniform spatial distribution, representing a high level of uncertainty regarding the robot's location.

In Figure 4, the blue arrow denotes the ground-truth robot pose within the simulation environment, while the red arrow represents the pose estimate obtained from the weighted mean of all particles. At this stage, the estimated pose differs significantly from the true robot pose because the particle weights are initially identical and no sensor observations have yet been incorporated into the filter.

This initialization strategy enables the Particle Filter to perform global localization, allowing the robot to determine its position without requiring an approximate initial pose estimate.

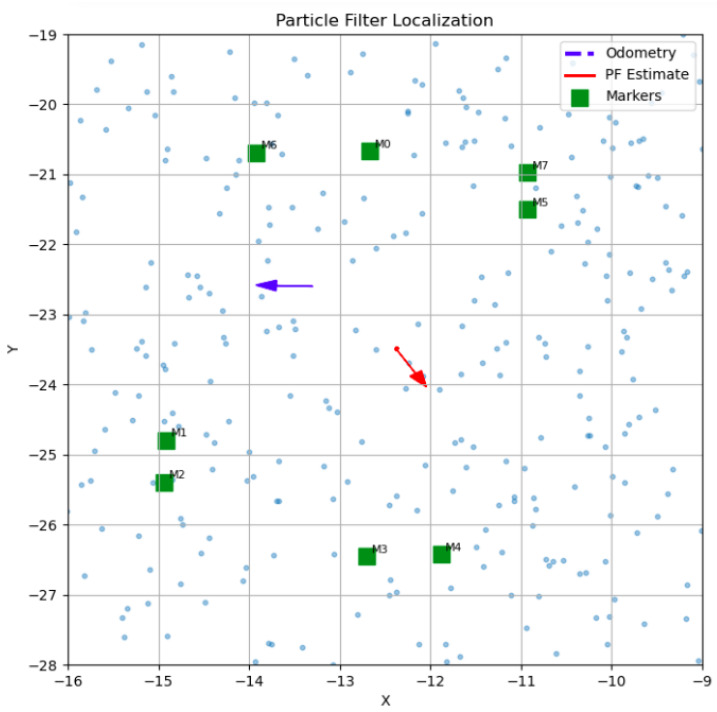

Figure 4: Initial particle distribution. The blue arrow indicates the ground-truth robot pose, while the red arrow represents the particle filter estimate obtained from the particle cloud mean. Since no observations have yet been incorporated, the estimate is not expected to coincide with the true robot pose.

It is worth noting that the initial particle filter estimate appears near the center of the environment despite the robot being located elsewhere. This behavior is expected and does not indicate successful localization. Because particles

are uniformly distributed and assigned equal weights during initialization, the weighted mean of the particle set tends to lie close to the geometric center of the map. Only after sensor observations are incorporated through the update step do particle weights become informative, allowing the estimate to gradually converge toward the robot's true pose.

5.2 Partial Convergence

After the first few sensor updates, the particle distribution begins to deviate from its initial uniform state. Observations obtained from visible ArUco markers allow the filter to assign higher weights to particles that better explain the measurements, while particles inconsistent with the observations gradually lose importance.

Figure 5 illustrates this intermediate stage of the localization process. Rather than being uniformly distributed throughout the environment, particles begin to form several distinct clusters. Each cluster represents a plausible hypothesis regarding the robot's pose. At this stage, the filter has successfully reduced the search space, but ambiguity still remains because multiple particle groups are capable of explaining the available observations.

The presence of multiple clusters is a common characteristic of global localization problems. As additional observations become available, incorrect hypotheses receive progressively lower weights and eventually disappear during the resampling process.

5.3 Final Converged State

As the robot continues to move and observe additional landmarks, the ambiguity between competing pose hypotheses is gradually resolved. Repeated prediction, weighting, and resampling cycles concentrate particles around the most likely robot pose.

Figure 6 shows the final converged state of the Particle Filter. The majority of particles are concentrated within a small region of the environment, indicating a significant reduction in pose uncertainty. The particle cloud has effectively collapsed to a single dominant hypothesis.

Although the odometry estimate (blue trajectory) exhibits noticeable drift due to accumulated motion errors, the Particle Filter successfully corrects these errors using landmark observations. Consequently, the estimated pose remains bounded and consistent with the map.

The final particle distribution demonstrates the ability of the proposed localization system to recover from large initial uncertainty and accurately estimate the robot pose using visual landmark measurements.

5.4 Video Demonstrations

In addition to the static figures presented in this report, several video demonstrations were recorded to illustrate the behavior of the localization system

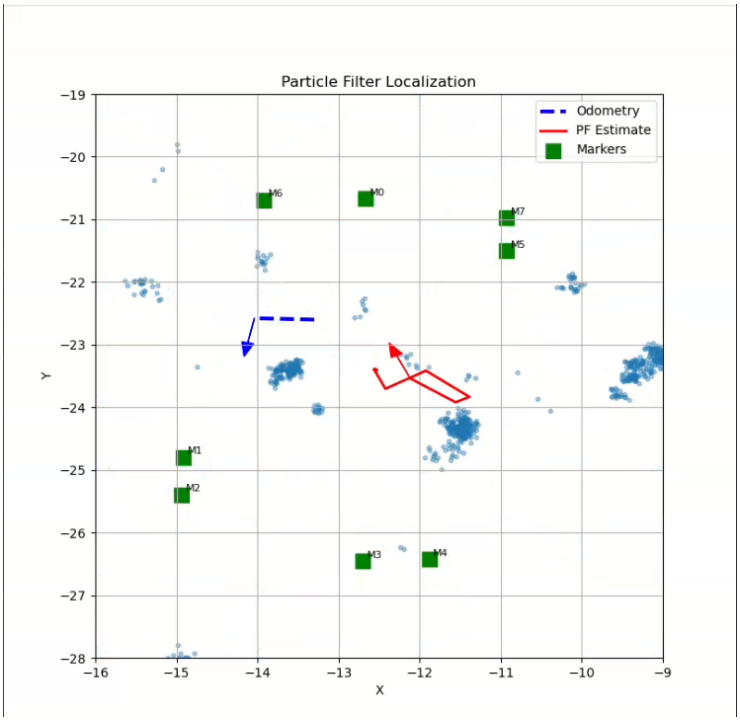

Figure 5: Intermediate localization stage showing multiple particle clusters. Several pose hypotheses remain consistent with the available observations.

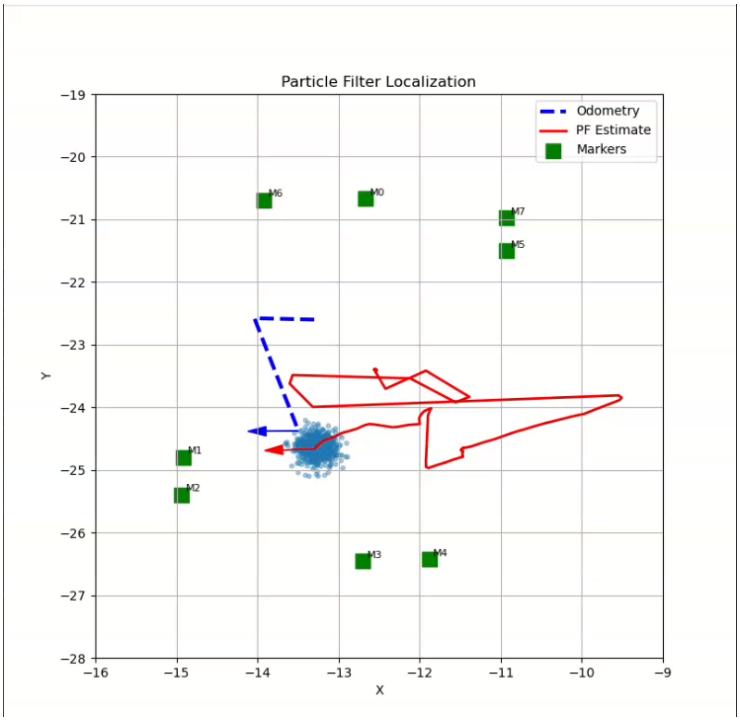

Figure 6: Final converged particle distribution. Most particles have concentrated around a single pose hypothesis, indicating successful localization.

during operation.

5.4.1 Simulation Environment Overview

The first video presents the complete simulation setup used throughout the experiments. The video includes a walkthrough of the Gazebo Harmonic environment, the ArUco marker placement, and the differential-drive robot model. It also demonstrates the RViz visualization environment, where the robot state, coordinate frames, and localization outputs can be monitored. Furthermore, the video shows how the robot is manually controlled through ROS 2 teleoperation commands and how both the ground-truth robot pose and the Particle Filter estimate are visualized simultaneously in the custom matplotlib-based monitoring interface.

The Overview Video Link:

`https://drive.google.com/file/d/1xFwwfk1jSon7Doqy54Ocl8aTugicboSv`

5.4.2 Algorithm Execution and Sensor Observations

The second video demonstrates the localization algorithm during runtime. The lower-left corner of the visualization displays the RGB camera stream received from the onboard camera. Simultaneously, the Particle Filter updates, particle cloud evolution, landmark detections, and pose estimates can be observed. This demonstration highlights the interaction between visual observations and the localization process, showing how landmark detections influence particle weighting and resampling.

The Algorithm Execution Video Link:

`https://drive.google.com/file/d/1AP6hA0hVbxjEkm_M3v9VaEfnztLc39Q9`

5.4.3 Robustness of The Particle Filter Algorithm

The final video illustrates one of the key properties of Particle Filter localization: recovery from large initial uncertainty. At the beginning of the experiment, the estimated robot pose may be significantly different from the true pose, sometimes even appearing outside the actual map boundaries due to the random initialization of particles. As the robot moves through the environment and observes additional ArUco markers, the Particle Filter gradually eliminates incorrect pose hypotheses and converges toward the correct location.

This experiment demonstrates the robustness of the localization system and its ability to solve the global localization problem without requiring an accurate initial pose estimate.

The Robustness of The Particle Filter Algorithm Video Link:

`https://drive.google.com/file/d/1bm4hPZ9SsMflVLshFWMg9Hv8_Zv7AAWP`

6 Conclusion and Code Availability

This report presented the design, implementation, and evaluation of a Particle Filter-based localization system for a differential-drive robot operating in a simulated warehouse environment. By integrating Gazebo Harmonic with ROS 2 Jazzy, the system successfully estimated the robot's pose using visual ArUco landmarks.

The experimental results demonstrate the robustness and accuracy of the implemented algorithm. Initializing the particles with a uniform distribution allowed the system to effectively solve the global localization problem without requiring a prior pose estimate. Despite the inherent drift associated with the odometry-based motion model, the multi-hypothesis sensor model reliably updated the particle weights based on range and bearing measurements, successfully steering the belief toward the true state.

Furthermore, the implementation of systematic resampling—triggered dynamically by a 50% Effective Sample Size (ESS) threshold—coupled with positional roughening (jitter), successfully prevented particle degeneracy. This ensured that computational resources were focused on high-probability regions while maintaining enough diversity to recover from tracking errors. Overall, the final converged state and the accompanying video demonstrations validate that the Particle Filter can consistently correct accumulated motion errors and track the robot with high precision.

6.1 Code Availability

The complete source code for this project is publicly available on GitHub at:

`https://github.com/sumeyraKoc/Particle-Filter`

The repository contains all the necessary implementation details, including the core ROS 2 nodes (`particle_filter.py`, `pf_visualizer.py`), the custom particle and resampling modules, configuration files, and launch scripts.